EXORCISM DOCUMENTATION

I apologize in advance, I am aware that the game is a fragmented mess, so to make this as easy as possible this is kind of a guide to where everything is and how some of the more convoluted things inside the game work. Another apology, halfway through this project I started learning c++ and for some reason I thought it would be cool to switch from naming conventions and sometimes just use my own made up naming conventions.

If you don't care about the organization and understand the code feel free to just ignore this whole thing.

Function Locations

Because GameMaker does not tell you where functions are defined, this is a list of where global functions and a few structs are defined.

- **Control**
  - **Group1**
    - scr_lighting
      - scr_calc_lights()
      - scr_lines(_line)
      - scr_lines_ip(_line)
      - scr_lines_ray(_line)
      - scr_draw_lights(_array, _pos, _vp)
      - scr_calc_size(_array)
      - scr_hiding_shadow(_other, _num)
    - Script1
      - scr_start()
      - scr_getroomsize(_id, z)
      - scr_light(_x, _y)
      - scr_lightbeam(_x1, _y1, a)
      - scr_opendoor(_x, _y)
      - scr_lightswitch(_x, _y, _cid, _obj)
      - scr_path_smooth(_path, _pts, _spd, _col)
      - scr_move_point(_xy, _col)
      - restart()
      - mvroom(_room)
  - items
    - scr_add_item(_obj)
    - scr_find_item_num(_obj)

        - scr_ready_items()
    - pathing
        - scr_path(Start, End)
        - find_location(col, _rm = -1)
        - path_follow(Pos, pathqueue, spd = 1.75)
    - salt
        - salt(_obj)
    - settup
        - vec2(_x, _y)
        - vec3(_x, _y, _z)
        - distance(_vec1, _vec2)
        - Vdir(_vec1, _vec2)
        - lengthdir_vec(_dist, _dir)
        - VA(_vec1, _vec2)
        - VE(_vec1, _vec2)
        - DLV(_vec1, _vec2, _w)
        - VecCreate()
        - scr_cid(VecP)
        - scr_MP(_start, _end, path)
        - scr_draw_connection(_start, _end)
        - scr_cdraw_path(_path)

Naming

Just a few naming things real quick.

- If anything starts with a ‘c’ that means custom, like ‘cid’. Because gamemaker already has a built in variable called id, which is basically a pointer, I could not use id as a variable name, hence cid - custom id.
- If anything starts with a ‘t’ that means test, such as _tlb which means test lightbulb, usually I used this when testing positions or what room something is in.
- Variables starting with _ was something I was taught to do for temporary variables, halfway through the project I stopped doing that because I realized there was no real reason to do it because all temporary variables are yellow, the _ has no real meaning.

Weird variable abbreviations

A lot of variables were abbreviated to the point where their purpose is unclear so here is this.

- _vp or vp = viewport
- FSW and FSH = floor size width and floor size height
- _surf or surf = surface
- _ord = an ordered array
- _off or off = offset
- _ls = light switch
- _lb = lightbulb
- m = matrix
- _col or col = collision
- Obj_game
    - el = enemy list
    - ae = active enemies
    - e = enemies
- pathing
    - _buf = buffer, not a data buffer, really more of an offset
    - VecPE = vector path end
    - _e = I dont think it means anything in particular
- salt
    - _f0c = floor zero circle
    - _f1c = floor one circle
    - _mid = midpoint
    - _cnum = candle number
- settup
    - _w = width
    - VecP = position vector
- Enemies
    - spdp = speed position
    - soff = shadow offset
    - _xso = x shadow offset
    - _yso = y shadow offset
    - _test = I just never named this, its basically sin(time)
    - _det = detail
    - traila = trail array
    - VecNS = vector new start
    - VecS = vector start

    - VecE = vector end
    - VecA = vector add
    - _tm = probably transformations, but im not sure
    - md = probably middle distance or distance multiplier
    - lkp = look point
- obj_player
    - cd = cooldowns
    - lhp = last hiding position
    - sel = selected
    - inv = inventory

Cid

Cid is how I track what room everything is in. This helps with some things like making sure the player can't turn on lights in another room, even if the lightswitch is close enough through a wall. This is also used in the code that calculates the enemies shadow, as it's an easy way to find all the light bulbs in the room. Cid is also important to the lighting system.

Cid is gotten from the cid of the obj_floor the object needing a cid is touching. Inside obj_floor's create event is an array called ctouching, which specifies which obj_floors are touching each other, this is necessary because some rooms are of abnormal shapes and cannot be covered by one floor without clipping into another room. So, inside the step event there is code that will increment room_ids, then give the room an id of the current value of global.room_id and then if its id is the same as another in the ctouching array, it will set its id to the other one, effectively making them the same room.

Enemies

Overview

Each enemy is slightly different but most of them share large portions of their code. They are also all drawn in code, although some use some sprites as well, such as obj_enemy_6's eyeball, or obj_enemy_4's mouth.
The code in the draw events is almost completely identical for enemies 1, 5, and 6 as well as somewhat 4. The only ones that stick out are 2, which only draws light beams, and enemy 3 which is made of many body segments. Also, obj_enemy_1_old is never used in the code.

This is a description of what each enemy is supposed to do/be:

- obj_enemy_1 is just a basic enemy, meant to chase the player when they spot them and otherwise just wander.
- obj_enemy_2 is supposed to circle around the outside of the building, looking in and hurting the player when they see them. I imagine it like a Gorilla with a bright lantern on it like an Angler Fish, hence the light.
- obj_enemy_3 is like a giant centipede, the player is warned when it is coming, after a short time it will then break through a window and run through the house very fast, before exiting out another window.
- obj_enemy_4 uses up hiding spaces, if the player tries to enter a hiding spot this enemy is occupying, the player will be hurt.
- obj_enemy_5 is just supposed to be a hazard. It is small and does very little damage and it just moves around randomly.
- obj_enemy_6 wanders around peacefully, but its eye is very sensitive to light, so it will attack the player if they turn on or off a light in the room it is in, it can also toggle lights itself.

Drawing enemies

Legs of enemies are drawn from the center of the body to a point that has a specified offset from the body. The position of the offset point is only updated on a timer, and it cycles through updating every other limb before updating the first another time, leading to a stepping effect, where the limbs will drag behind the bug but snap back to it before going too far. All rotations and such are done by setting a matrix with the rotations and position, drawing the bug and body, then returning to an identity matrix.

obj_enemy_2 light

obj_enemy_2 is only visible through its light, makeshift rays that stop when they collide with anything on their collision list. The scr_light(_x, _y) function in obj_enemy_2's draw event is declared in Script1. Looking back now I see that scr_light(_x, _y) and scr_lightbeam(_x1, _y1, _a) could have been greatly optimized but oh well. The way it works is each lightbeam will determine an x and y distance to move for each of their "steps" it will then in a for loop check the current position, see if its colliding, if not it will draw a line that is later blurred as the light, and it will add the determined x and y to its position until it hits something or runs out of iterations.

obj_enemy_3
obj_enemy_3 is drawn in basically the same way as the others except it is drawn in segments. It has an array (traila) of segment positions and basically each segment gets pulled towards the one in front of it if it is too far, but never pushes away from the one in front of it, leading to a dragging / coiling effect as it runs and attacks the player. This is done through the trail function in its create event.

Enemy shadows
I forgot to give most enemies shadows, but for obj_enemy_1 and obj_enemy_6, here is this. In the step event it finds all the positions of the light bulbs in the room and averages them together into the variable _lp for where approximately the light causing the shadow of the enemy should be projected from. It then calculates an x and y for soff (shadow offset). Then inside the draw event it draws a dark transparent copy of the body offset by soff, but does not offset the position of the points the legs are drawn towards so the enemies shadows do not make them look like they are floating.

Path Finding

GameMaker has its own built in path finding functions, so I did not need to implement my own algorithm, though I probably should have. From my experience the path finding functions are finicky at best and consistently crash the game for absolutely no reason at worst. I wanted the bugs to be able to follow the player through the house naturally, not walk out a window in the attic and then walk in a window in the basement, (except for the one bug that's supposed to do that) I wanted the bugs to have to go up and down the stairs as well. That's a problem, because inside the room all three floors of the house are just right next to each other. The only way to fix that is to put invisible walls around the outside of every floor, this meant that the bugs could not find a way through the floors and would crash the game every time they wanted to move, the solution? My makeshift path system that searches for ways to get to other spaces that aren't connected, like stairs.

Enemy states
obj_enemy_1, obj_enemy_3, and obj_enemy_6 use path finding. All of them have different states, in the different states the target of their path finding will be different.

obj_enemy_1

- (state = 0) Wandering, the bug will move to a random position on a random floor using the find_location() function with an unspecified cid.
- (state = 1) Searching, the bug has recently spotted the player but has lost track of them, either because they are hiding or have gone out of the line of sight of the bug for too long. It will move to a random position in the room the player is in with the find_location() function and specifying a room that has the same cid as the player.
- (state = 2) Chasing, the player is currently in the line of sight of the bug and the bug is chasing the player, the bing will pathfind to the player's position.
- (state = 3) Food, the bug has spotted a piece of meat the player left out and will pathfind to it, it prioritizes this over the player.

obj_enemy_3

- (state = 0) Waiting, the bug will be stuck at a path at (0, 0), the bug will not move until the time for the player to hide has run out.
- (state = 1) Running, the bug will spawn at one window, then pathfind through the house to another window and run through it very fast.
- (state = 2) Chasing, the player is currently in the line of sight of the bug and the bug is chasing the player, the bing will pathfind to the player's position.

obj_enemy_6

obj_enemy_6 only has two states so instead of an int in charge of the state, it's controlled by a bool.

- (attacking = false) Wandering, the bug will move to a random position on a random floor using the find_location() function with an unspecified cid.
- (attacking = true) Mad, the player turned on or off the lights while in the same room as the bug and the bug is now mad, it will chase the player.

Non-pathfinding enemies

obj_enemy_4 does not move so it does not use any pathfinding. While obj_enemy_2 and obj_enemy_5 do move, obj_enemy_5 moves randomly, and

obj_enemy_2 just has a square path that circles around the outside of one of the floors of the house.

Path queue and path handling
For the enemies that do use my pathfinding, they all have a pathqueue, a list of paths for them to follow. The purpose of this is so that instead of pathfinding from floor one to floor three, finding no path, and crashing my game, the bug will find a path to stairs leading up to floor two, then a path leading to stairs up to floor three, then a path to wherever they wanted to be on floor 3. Along with the pathqueue are two functions, switch_queue(paths) which is used to move onto the next path once the end of the current one has been finished and path() which handles adding paths to the pathqueue based on the state.

Path creation
The paths are created using the scr_path(Start, End) function, which compares the floors of the start and the end which forms the “bigPath” then breaks each floor into a normal path, putting in predefined positions of stairs. The normal paths are made using GameMakers mp_grid_path() function, and the global.grid, which is a grid filled with all the objects the pathfinding function should avoid.

The grid is created in the function scr_start() which is defined in Script1 and then called in Alarm 0 of obj_game. The function adds all instances of obj_enemy_wall and obj_hiding_spot to the grid as things that the enemy cannot pass.

Unused path features
An early version that I don’t *think* is actually used in the code is defined in settup script with scr_MP(_start, _end, path) and scr_draw_connection(_start, _end). This code worked mostly the same as it does now, just coded much worse. Inside Script1 there is a function scr_path_smooth(_path, _pts, _spd, _col) which was another experimental thing that I did not use in the final version, as the name suggests it smoothed paths. The intent was to make the path appear more natural but the result was enemies clipping through walls around corners, so I never used it.

Lighting

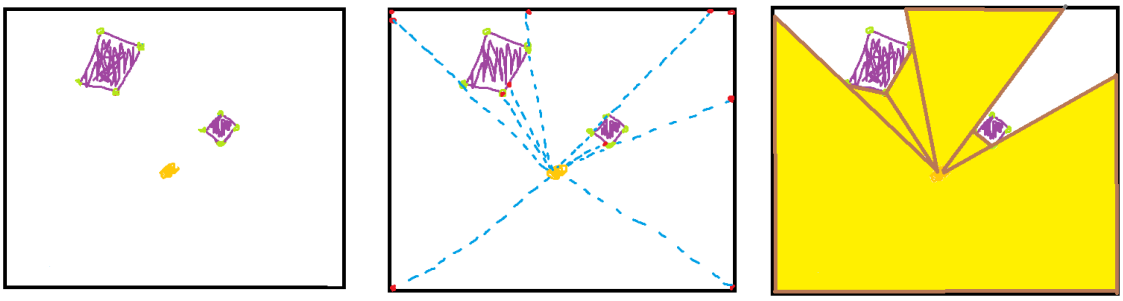
This is definitely the most fragmented part of the code, and also the reason why I decided it was best to make this document. I wanted the lighting to look good, so I ended up redoing most of it at least four times. There are two main parts, the pure black shadows, like the ones that cover rooms you're not in and that show up when the player is hiding, and the more realistic softer shadows that affect the saturation as well.

Solid shadow

When in a room, all other rooms are covered. The function scr_getroomsize(_id, z) will give an array of points that can be used to draw black boxes that cover the other rooms. The function is used around line 85 of the obj_player step event. The points are then used inside the player's draw event around line 10 to draw the hard shadows. The other solid shadows are the hidden shadows that appear when the player is hiding. This is handled with scr_hiding_shadow(_other, _num) which basically just creates the shapes of the shadows with vertices then draws the shapes.

Other shadows

When the game starts every obj_light will call scr_calc_lights(). Scr_calc_lights() goes through every object in global.col and adds a vertex for each of its corners to an array, it also adds one for the corner of each room. It then uses scr_lines_ip() to test and draw a line from the lightbulb towards the point as far as it can until it hits something or goes outside of the room. Those hits are added as new points to the array, then the array is sorted by angle around the lightbulb and returned.



Back in the lightbulbs draw event, the points are then sent into the function scr_draw_lights(_array, pos, _vp). This function starts a primitive and adds all the points to it, effectively making the points into a solid shape of the light. Shd_lighting2 is then turned on, this will make the alpha of the sprite get fainter the farther it is from the lights position, it is also given the color for some reason,

that's not important as I don't think the color is ever used. This is then drawn onto a surface, turned into a sprite, then returned, effectively making the alpha mask of the light.

Separately from all of this the light bulbs will also create a sprite that is a kind of “screenshot” of the room, it does by using sprite_create_from_surface() on a portion of the the application_surface, the surface that game maker draws everything to by default and is what will be shown on the screen. For memory reasons it uses the bounding box of the points received from scr_calc_lights() to know what to take a screenshot of. Finally, it draws the alpha mask with shd_lighting3 on which will set the color of the mask to a saturated version of the normal background.

Obj_shade_control is in charge of darkening all the rooms that don't have their lights on. It basically sets the drawing mode to one that will darken everything, but keep the saturation high, then it draws a dark rectangle over the entire room. How dark it is is based on the lights, so unilluminated parts of rooms that have the lights on are brighter than they normally are when the lights are off to mimic a sort of ambient lighting.

Gameplay

Enemy spawning

Enemy spawning is handled in the obj_game. The variable ‘el’ contains a list of all enemies and a number that is their supposed spawn rate. The game has its own time system that keeps track of how many in-game minutes and hours have passed. There were originally supposed to be multiple difficulties and the harder ones lasted longer, but I never really added that.

Inside obj_game’s Alarm 1 event it will trigger event_user(0) once at every half hour and another time every hour, event_user(0) is in charge of deciding which enemy spawns, where it spawns, adding it to the ‘e’ array and setting how long it will live for.

Enemy despawning

Enemy despawning is entirely handled within obj_game’s step event, it will cycle through the ‘e’ array and decrement the timer in charge of how long they live,

once that reaches 0 the instance is destroyed and the enemy is removed from the array.

Items

Inside the ‘items’ script, all of the item objects are added to the global.items array, which for each item stores a reference to the object itself, a reference to the sprite of the object, the name of the object as a string, and a number that functions as the items id. With the exception of obj_meat and obj_pot the only within each object is a number that is its id, and a draw event that draws itself. All functions of items are at around line 390 inside obj_player’s step event. All the hotbar code is also inside the player, the only thing the players inventory stores is the number that is the id of the items they are holding.

Salt

One of the major things required for beating the game is making both the salt circles and arranging the items on them in the correct way, a salt circle in the basement with three candles and a bug, and a salt circle on the middle floor with the silver cross in it. The weird thing here is detecting if the salt is forming a circle, or if the player just drew a scribble with it. All of this code is in the singular salt(_obj) function inside the salt script.

The function starts by iterating over every obj_salt, if the distance to the next piece of salt is close enough, it keeps checking, if it is far enough it ends the ‘chain’ of salt and adds it to an array then goes on to checking the remaining salts. Once all salt chains have been found it checks through them and averages their positions, it then checks every piece of salts distance from that point, if all the distances are roughly equal than its a circle and is added to an array of correct salt circles.

After all salt circles have been found the code then checks for win conditions, like whether the three candles and the bug are closer to the center point than the radius of the circle. If both salt circles are good then it will start the win ‘animation’ and the player will win.